

How does a program ask the
operating system for help?
OR What happens behind
the scenes when a program
opens a file?

System Calls

Bridge between user application and OS.

Learning Objectives



Understand the importance of system calls.

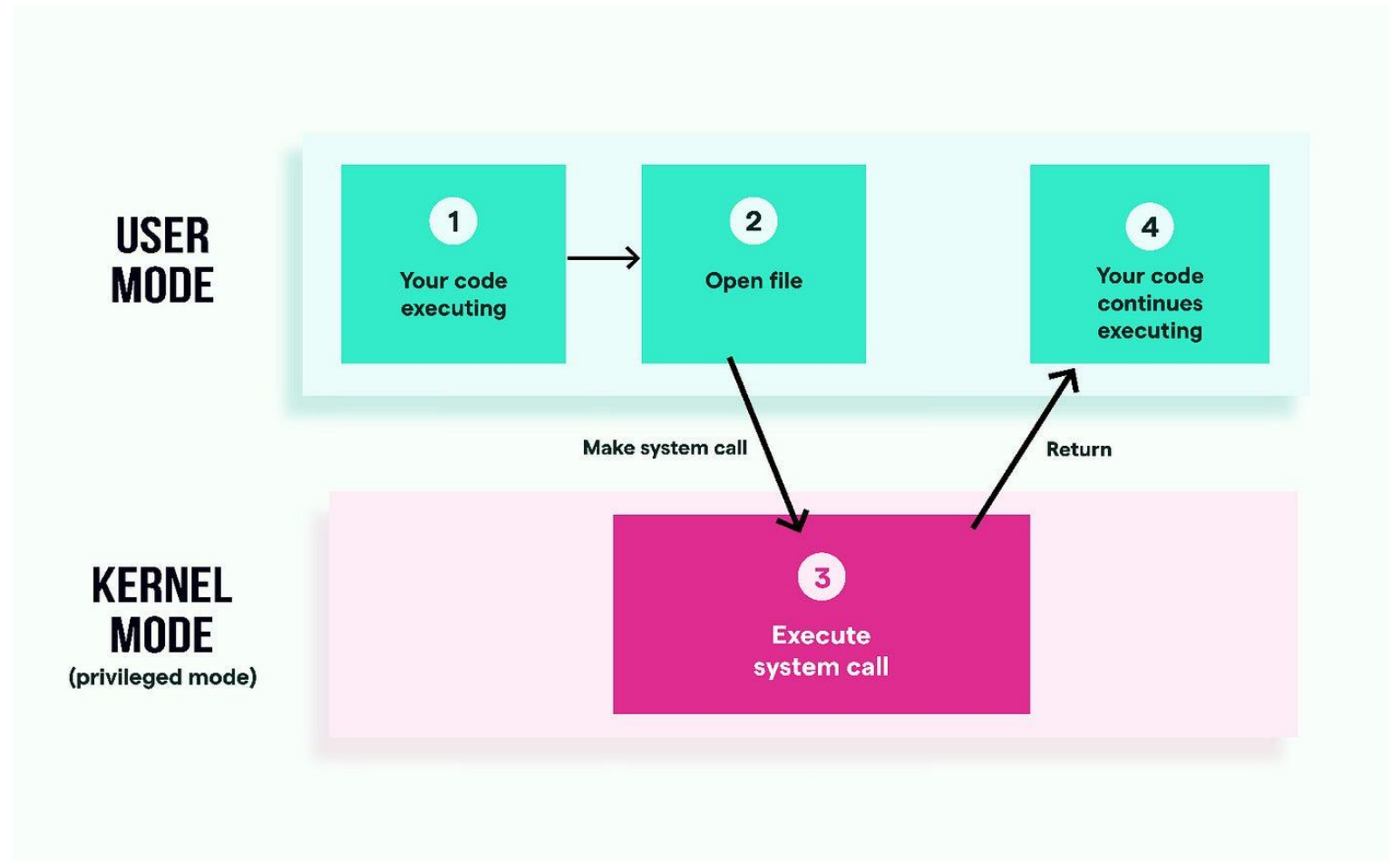


Differentiate between API and system call interface.



Analyze the flow of system call invoke.

Illustration



System Calls



System calls are the **interface between user programs and the operating system**



They allow programs to **request services from the OS kernel**



System calls provide **controlled access** to hardware and system resources



They are executed in **kernel mode**, not user mode

How does a user program
know which system call to
invoke and how to invoke it?

System Call Interface



The **System Call Interface (SCI)** is the **mechanism that connects user programs to system calls**



It provides a **standard method** for invoking OS services



Acts as an **abstraction layer** between applications and the kernel

Role of the System Call Interface



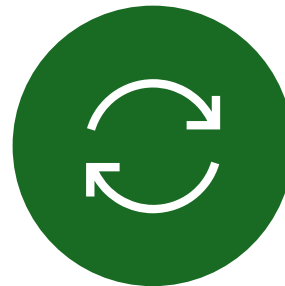
Translates **application requests** into system calls



Hides OS implementation details from users



Ensures **correct and secure invocation** of system calls



Provides portability across different systems

System Call Number



Each system call is assigned a **unique number**



Used by the kernel to identify the requested service

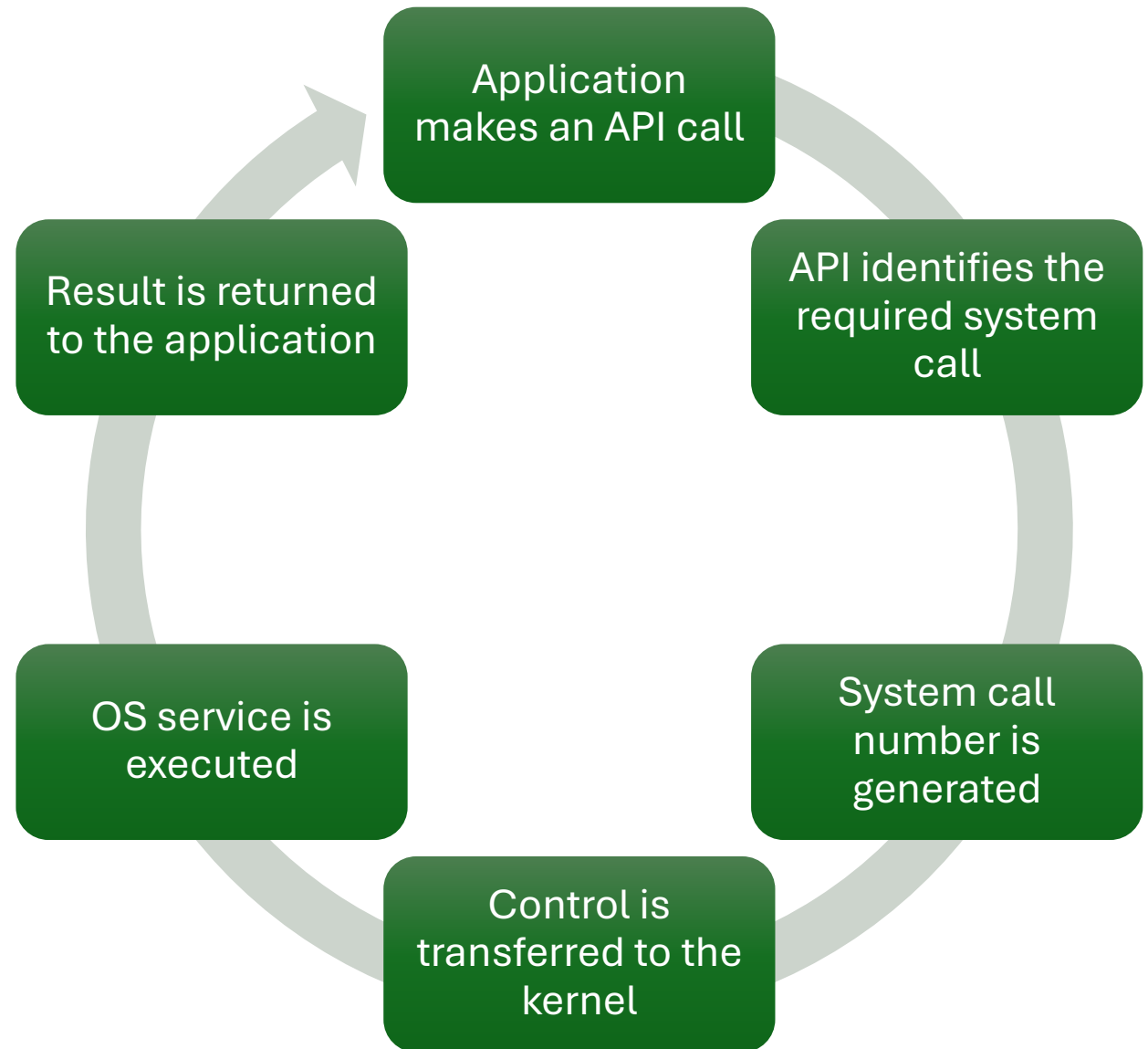


Stored in a register or memory location



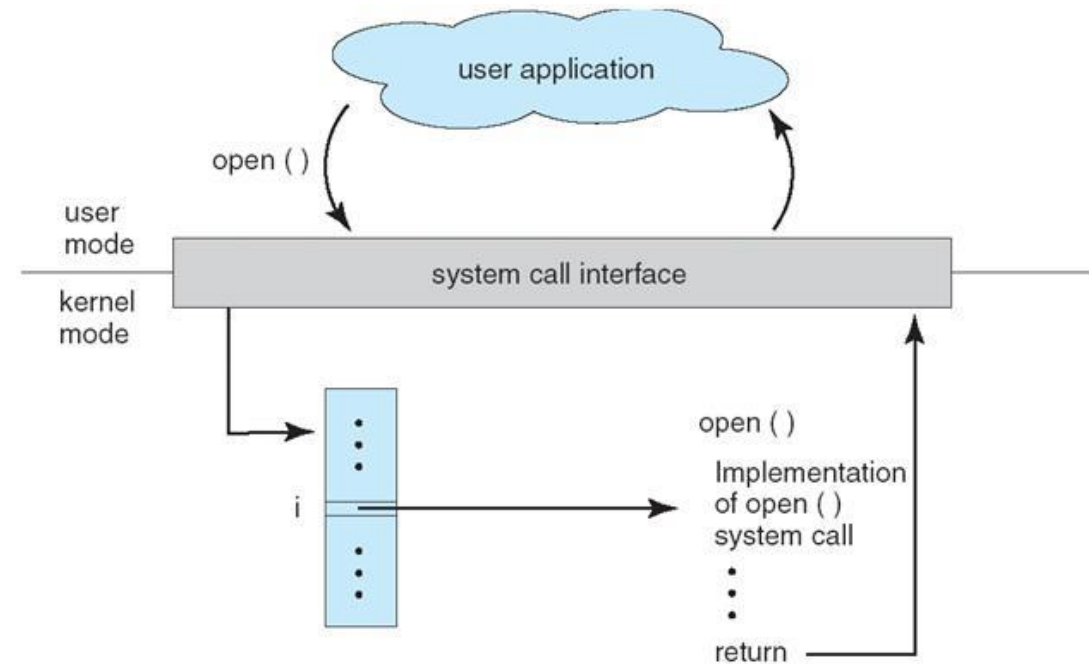
Enables fast and efficient dispatching

How the System Call Interface Works

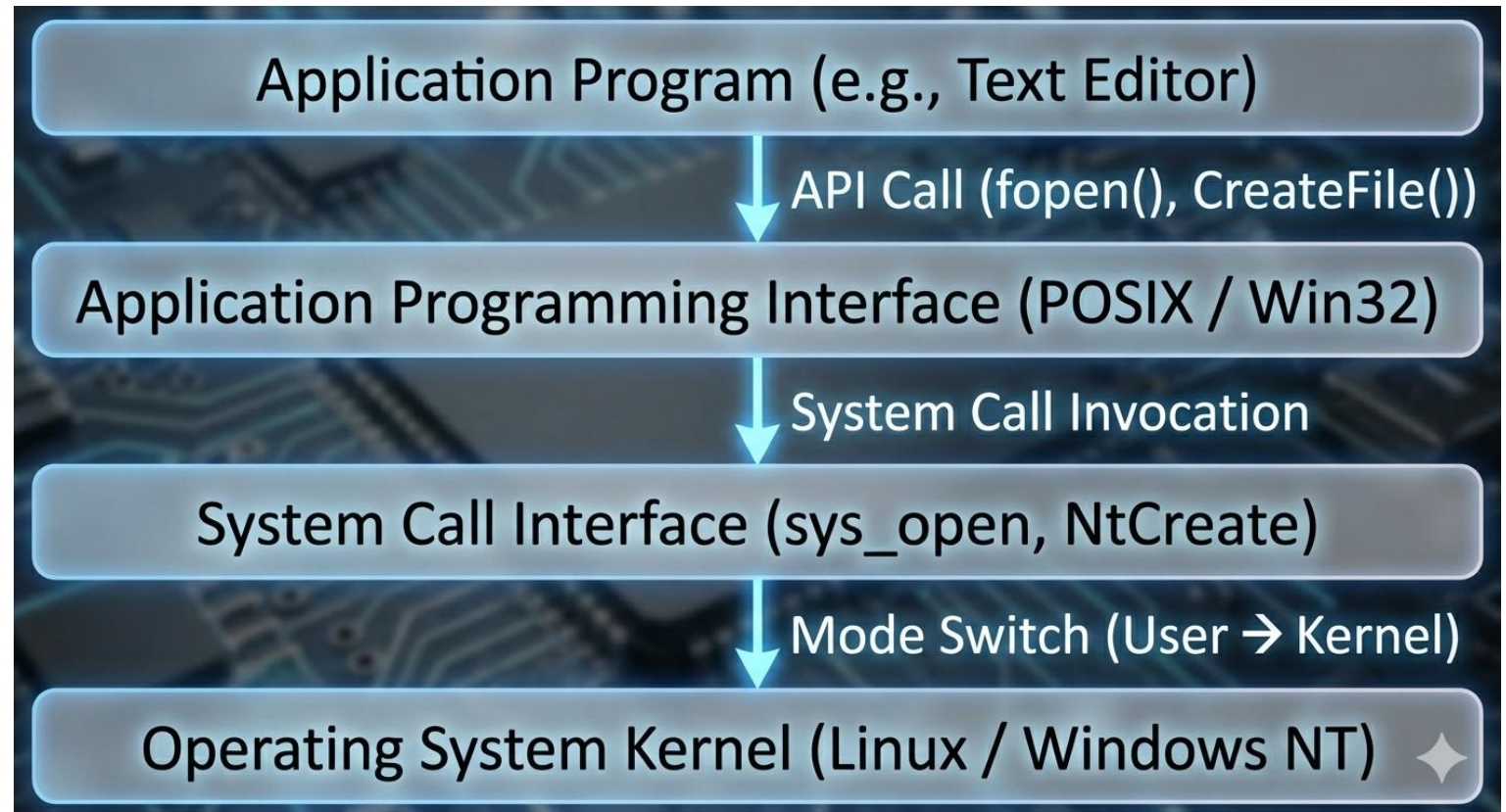


System Call Interface Components

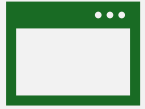
- Application programs
- Application Programming Interface (API)
- System Call Interface
- Operating System Kernel



Illustration



Application Programs



Web browser (Google Chrome, Firefox)



Text editor (Notepad, Vim)



Media player (VLC)



Command-line tools (ls, dir))

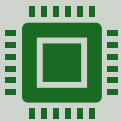
Application Programming Interface (API)



POSIX API (Linux/UNIX): `printf()`, `fopen()`, `malloc()`



C Standard Library (glibc): `open()`, `read()`, `write()`



Win32 API (Windows): `CreateFile()`, `ReadFile()`, `CreateProcess()`

System Call Interface



Linux system calls: `sys_read`, `sys_write`, `sys_fork`



System call numbers (Linux): `SYS_read`, `SYS_write`



Windows Native API: `NtCreateFile`, `NtReadFile`

Operating System Kernel



Linux Kernel

Windows NT Kernel

macOS XNU Kernel

Example

Layer	Example
Application Program	Notepad, Chrome
API	POSIX, Win32
System Call Interface	sys_read, NtCreateFile
OS Kernel	Linux Kernel, Windows NT

Analogy



Application → Customer



API → Reception desk (friendly functions)



SCI → Security gate (checks and routes requests)



Kernel → Restricted office (does the real work)

Important clarification



SCI is NOT something programmers call directly.



Programmers call APIs (fopen(), CreateFile())

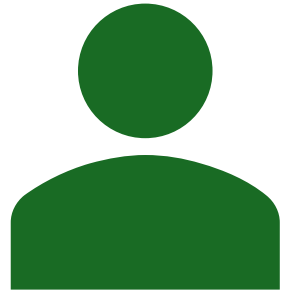


APIs internally invoke system calls



Those system calls enter the kernel through SCI

Precisely



API = what you see



SCI = how the OS receives it

Code Example

Application Code (User Space)

c

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    fd = open("data.txt", O_RDONLY);    // API call
    close(fd);                          // API call
    return 0;
}
```

What Really Happens (Behind the Scenes)

Application Program



`open()`

← API (library `function`)



glibc `wrapper`



SCI (`System Call` Interface)



`sys_open()`

← Actual `system call` (kernel)



Kernel opens the file

Key Points

`open()`

→ **NOT a system call**

→ User-space API function

`sys_open()`

→ **REAL system call**

→ Executes in kernel mode

System Call Interface (SCI)

→ Entry mechanism into the kernel

→ Switches CPU from user mode to kernel mode

Summary



System Call Interface is the **gateway to OS services**



It separates user programs from kernel implementation



APIs rely on the SCI to invoke system calls safely